Multinomial Naive Bayes

Text classification with word counts

The problem

- Input: word counts per document
- Output: class, e.g. spam / ham
- Vocabulary is large and sparse

Model

$$\log P(c \mid x) = \log P(c) + \sum_{w} x_w \log \theta_{cw} + \text{const}$$

- θ_cw = share of class c's words that are w
- Score is linear in the counts

Laplace smoothing

$$\theta_{cw} = \frac{N_{cw} + \alpha}{\sum_{w'} N_{cw'} + \alpha V}$$

- Unseen word → probability 0 → log = −∞
- Add α (usually 1) to every count

MultinomialNB

```
class MultinomialNB:
    def __init__(self, alpha=1.0):
        self.alpha = alpha

    def fit(self, X, y):
        X, y = np.asarray(X, float), np.asarray(y)
        self.classes_ = np.unique(y)
        counts = np.array([X[y == c].sum(0) for c in self.classes_]) + self.alpha
        self.log_likelihood_ = np.log(counts / counts.sum(1, keepdims=True))
        self.log_prior_ = np.log(np.array([np.mean(y == c) for c in self.classes_]))
        return self

    def predict(self, X):
        scores = np.asarray(X, float) @ self.log_likelihood_.T + self.log_prior_
        return self.classes_[scores.argmax(1)]
```

Complexity & trade-offs

- Train O(nnz) — one pass over counts
- Predict: one sparse matrix product
- Ignores word order and correlation
- Great for spam, topics, sentiment baselines

Recap

- Per-class word distributions
- Laplace smoothing prevents zero probabilities
- Log-space scores = linear model
- Fast and strong for text